Azure ML Environments

Method 1: Lightweight Conda (Pip Packages)

When your machine learning job only requires standard Python packages (e.g., via `requirements.txt`) and no underlying operating system-level dependencies, the lightweight Conda approach is the fastest and easiest method to configure an Azure ML Environment.

Step 1: Create the conda.yml Configuration

Create a file named `conda.yml` in your project directory. This file instructs Azure ML to use pip to install your required Python packages onto a standard Microsoft base image.

```
name: custom-job-env
channels:
  - conda-forge
dependencies:
  - python=3.8
  - pip
  - pip:
    - -r requirements.txt
```

> **Tip:** Ensure your `requirements.txt` file is located in the same directory as this `conda.yml` file.

Step 2: End-to-End Submission Example

Below is a complete, end-to-end Python script that connects to your Azure ML workspace, registers the custom environment using your `conda.yml`, defines a Command Job, and submits it to your compute cluster.

```
from azure.identity import DefaultAzureCredential
from azure.ai.ml import MLClient, command
from azure.ai.ml.entities import Environment

# 1. Connect to the Azure ML Workspace
ml_client = MLClient.from_config(credential=DefaultAzureCredential())

# 2. Define and Register the Environment
custom_env = Environment(
    name="my-lightweight-pip-env",
    description="Environment built with requirements.txt and conda.yml",
    # Using a standard Microsoft Ubuntu base image
    image="mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04:latest",
    conda_file="conda.yml" # Path to your local conda.yml file
)

registered_env = ml_client.environments.create_or_update(custom_env)
print(f"Registered Environment: {registered_env.name} v{registered_env.version}")

# 3. Define the Command Job
my_job = command(
    code="./src",               # Local folder containing your training scripts
    command="python train.py",  # The entry script you want to run
    compute="my-cluster-name",  # Replace with your actual compute cluster name

    # Reference the environment we just registered
    environment=f"{registered_env.name}:{registered_env.version}",

    display_name="pip-env-training-job"
)

# 4. Submit the Job
submitted_job = ml_client.jobs.create_or_update(my_job)
print(f"Job submitted! View it here: {submitted_job.studio_url}")
```